Welcome to your next lesson: Modeling relationships: one-to-one and one-to-many.

I'm getting your personalized lesson ready — using what you've learned, what comes next, and how to help you understand it better.

Step by step, you're moving forward.

Imagine a simple situation 🐶🐱:

You have one pet owner (you), and you have two pets: a dog and a cat.

We want our program to show that this one owner is connected to both pets, not just that three objects exist separately.

Visually:

You (owner)
↘ dog
↘ cat

Those arrows are the relationships.

Quick check, just by feeling:

Does that picture feel like…

one-to-one
or one-to-many

between owner and pets?

You can just answer: "one-to-one" or "one-to-many".

One-to-many

Exactly — one-to-many 🎯

One owner → many pets.

Let’s make that more concrete with a tiny diagram:

```
PetOwner Ron
↘ Pet Fido
↘ Pet Whiskers
```

Here, Ron is one object, and he’s connected to two different Pet objects.

That “fan out” is what makes it one-to-many.

Next, I want to show you how we represent a single connection in code (a simple one-to-one) before we layer on “many”. That way the jump is small.

We’ll start with:

one PetOwner
one Pet
and give the owner an attribute that refers to the pet.

Does that feel okay as a next step?

Yes, let's try it

Nice 🙌

Let’s build a simple one-to-one first.

Real world:

You (Ron) have one vet clinic membership card.

Program version:

one PetOwner object
one Pet object
the owner has an attribute that points to that pet

Visually:

PetOwner ——→ Pet

owner.main_pet is the arrow.

In Python, that could look like (just to see the idea):

Python
```
class Pet:
    def __init__(self, name):
        self.name = name

class PetOwner:
    def __init__(self, name, main_pet):
        self.name = name
        self.main_pet = main_pet  # relationship attribute
```
Key idea:

A relationship attribute is just an attribute that stores another object.

Before we touch code in the editor, quick check:

In this setup, which object is storing the relationship?

You can answer in words like: “the owner stores it” or “the pet stores it”.

self.main_pet

Great, you’re reading the code exactly how I hoped 🎯

self.main_pet is the attribute name, and it lives inside the PetOwner object.

So in plain words: the owner stores the relationship to the pet.

PetOwner knows: “this is my main pet”
Pet doesn’t know who owns it (in this simple version)
Now let’s stretch this to the many case.

Same idea, but now:

one PetOwner Ron
three pets: Fido, Whiskers, Luna
Ron needs some way to know about all three

Visually:

PetOwner (Ron)

↘ Pet (Fido)

↘ Pet (Whiskers)

↘ Pet (Luna)

The important idea:

One object can be connected to multiple other objects.

That’s a one-to-many relationship.

I won’t bring in lists yet — just the concept: owner “has” many pets.

Does this picture of one-to-many between PetOwner and several Pet objects make sense so far, conceptually?

Yes, that makes sense

Awesome 👍

Now let’s bring in responsibility slowly.

In our model:

PetOwner knows which pets they have
Each Pet knows things about itself (name, hungry or not, etc.)

So we’re saying:

The owner is responsible for managing the relationship

Each pet is responsible for its own state (like hunger)

Concrete example:

Pet might have is_hungry
PetOwner might have a way to tell a pet to eat

So a relationship-driven state change looks like:

We call a method on the owner
The owner uses its relationship to reach a pet
The pet’s state changes (for example, is_hungry becomes False)
Does that chain of responsibility (owner knows who the pets are, pets manage their own state) feel clear so far?

Yes, that’s clear

What do you think? 👇

In our PetOwner → Pet setup, who should store the relationship to keep responsibilities clear?

A.
The PetOwner, because it manages which pets it has.

B.
Each Pet, because pets should manage their owners.

C.
Both PetOwner and Pet, always in both directions.

D.
Neither, the relationship should be global.

Nice work, Ron 🎉 You picked exactly the right answer.

Why PetOwner should store the relationship:

The owner's job in our model is to manage which pets they have
So it makes sense that the owner object keeps the "list of connections" to its pets
Each Pet focuses on its own state (name, hunger, etc.), not on coordinating all owners

So:

PetOwner stores the relationship,

Pet manages its own data.

Next small step: I want to walk through a tiny Python example where a method on PetOwner changes a Pet's state (like feeding a pet) through that relationship.

Ready to look at that example next?

Yes, show me the example

Great, let's build it step by step 🐕🍖

We’ll keep it very small:

Pet has a state: is_hungry
Pet has a method eat() that changes that state
PetOwner has a relationship attribute that points to a pet
PetOwner has a method that uses that relationship to feed the pet

Conceptually:

PetOwner.feed_main_pet() → calls → Pet.eat() → changes Pet.is_hungry

So the call starts at the owner, but the state change happens in the pet.

Before I show the code, quick check:

In that chain above, whose state actually changes — the owner’s, the pet’s, or both?

the pets

Exactly — the pet’s state changes ✅

The owner starts the action (calls a method)
The relationship lets the owner reach the pet
The pet flips is_hungry from True to False

So this is a relationship-driven state change: using the connection between objects to change one of them.

Now I’d like to show you this in actual Python, then we’ll trace it.

Here’s the code:

Python
class Pet:

```
    def __init__(self, name):
        self.name = name
        self.is_hungry = True

    def eat(self):
        print(self.name, "is eating...")
        self.is_hungry = False


class PetOwner:
    def __init__(self, name, main_pet):
        self.name = name
        self.main_pet = main_pet   # relationship to a Pet

    def feed_main_pet(self):
        print(self.name, "is feeding", self.main_pet.name)
        self.main_pet.eat()        # use relationship to
change pet state


# setup
fido = Pet("Fido")
ron = PetOwner("Ron", fido)

print("Before feeding, Fido hungry?", fido.is_hungry)
ron.feed_main_pet()
print("After feeding, Fido hungry?", fido.is_hungry)
Don’t run it yet — just look at the flow.


Step-by-step arrows:


ron.feed_main_pet()
inside that, we call self.main_pet.eat()
that is really fido.eat()
fido.eat() sets fido.is_hungry = False
To check your understanding:


In the line self.main_pet.eat() inside PetOwner, which object
is self at that moment — the owner or the pet?

self is refereing to the the instance of petowner, which has
a refernce to a pet and calls its eat method

Perfect explanation, Ron 💯
```

Inside feed_main_pet, self is indeed the PetOwner instance (ron).

Because self is the owner:

self.main_pet is the related Pet object (Fido)
self.main_pet.eat() means: “tell my pet to run its eat() method”

So the call is owned by the owner, but the state change happens in the pet (fido.is_hungry becomes False). That’s exactly the data flow I wanted you to see 🎯

Now I want to check your feel for the relationship direction.

Imagine this setup (same idea, just words):

ron is a PetOwner
fido is a Pet
ron.main_pet points to fido
fido does not have any attribute that points back to ron

Question (conceptual, not code):

Is there still a relationship between ron and fido, or do they count as unrelated objects in the program?

You can answer in your own words.

there is a relationship in that the Pet is an instance attribute of petowner. however the pet has no idea who its owner is (itll take food from anyone), it has no reference upwards to petowner

Exactly right, and I love how you phrased it 👌

In the program, there is a relationship because
Pet is stored as an instance attribute on PetOwner (ron.main_pet).
But the direction is one-way:

Owner → knows the pet
Pet → doesn't know the owner (no attribute like owner)

So they are not just two random objects; they're connected, but only from the owner's side. That's a nice, clear mental model 💡

Next, I'd like to check one more idea: one-to-one vs one-to-many using a quick multiple-choice question.

Ready for a small quiz question on that?

Ready for the quiz

What do you think? 👇

PetOwner has three pets, and the program models the connection from the owner to each pet. What relationship is this?

A.
One-to-one

B.
One-to-many

C.
No relationship

Nice job, Ron — you nailed that ✅

One owner
Connected in code to three pets
Arrows all go: PetOwner → Pet, Pet, Pet

That's exactly a one-to-many relationship: one PetOwner object is related to many Pet objects. The relationship is real in the program because the owner stores those connections (through its attributes), not just because multiple pets exist.

You’ve just:

Shown you understand one-to-one vs one-to-many
Chosen correctly where the relationship should live (PetOwner)
Tracked how a method on one object can change the state of a related object

That’s the full set of skills for this lesson 🎯

If you want, we can now:

briefly revisit any part (relationships, direction, or state changes), or
you can head to the next lesson and build on this.